RES POLYTECHNIC, BANGALORE-91

CIE Theory Test I - Model Answers

Programming Concepts in Mechanical Engineering | Course Code 25ME34I | Marks 50 | Semester III

SECTION I

Q1 a) Flowchart – factorial using a loop (7 marks)

Algorithm: 1. Start 2. Read n 3. Set fact = 1, i = 1 4. If i ≤ n, go to step 5, else go to step 7 5. fact = fact × i 6. i = i + 1, go to step 4 7. Print fact 8. Stop

```
    ( START )
        |
   [ INPUT n ]
        |
  [ fact = 1, i = 1 ]   <-- Initialisation
        |
   < i <= n ? > --- No ---> [ OUTPUT fact ] --> ( STOP )
        |
       Yes
        |
  [ fact = fact * i ]
  [ i = i + 1 ]
        |
        +---------> back to decision
```

Symbols: oval = start/stop, parallelogram = input/output, rectangle = process, diamond = decision.

Q1 b) Dividing a problem into sub-tasks (6 marks)

Beam-load calculator can be split into: (1) input span and loads, (2) calculate support reactions, (3) calculate shear force and bending moment, (4) display results.

- **Easier to write:** each small task is simple and can be written separately.
- **Easier to test:** each sub-task is tested on its own with known values (e.g. reaction of a simply supported beam = W/2).
- **Easier to debug:** an error is traced to one sub-task instead of the whole program.
- Sub-tasks can be reused and shared among team members.

Q1 c) Machine name, rated power and running status (6 marks)

```
name = "Lathe"      # str
power = 7.5         # float
running = True      # bool
```

Variable	Type	Justification
name	str	Name is text
power	float	Power can have decimals (7.5 kW)
running	bool	Only two states: running (True) / not running (False)

Q1 d) input() function (6 marks)

- `input()` displays a prompt, waits for the user to type and press Enter.
- It **always returns a string (str)**, so it must be converted using `int()` or `float()` for calculations.

```
length = float(input("Enter length (mm): "))
breadth = float(input("Enter breadth (mm): "))
area = length * breadth
print("Area of plate =", area, "mm^2")
```

OR

Q2 a) Flowchart – smallest of T1, T2, T3 (7 marks)

```
    ( START )
        |
  [ INPUT T1, T2, T3 ]
        |
   < T1 <= T2 and T1 <= T3 ? > -- Yes --> [ small = T1 ]
        | No                                    |
   < T2 <= T3 ? > -- Yes --> [ small = T2 ]    |
```

```
      | No                          |          |
[ small = T3 ]                      |          |
      +---------------------------+----------+
                    |
            [ OUTPUT small ]
                    |
                ( STOP )
```

Q2 b) Four features of Python for engineering (6 marks)

1. **Simple, readable syntax** – fewer lines than C++/Java; no need to declare variable types.
2. **NumPy** – fast array/matrix operations (stress, stiffness matrices).
3. **Matplotlib** – easy plotting of graphs and charts.
4. **Interpreted and interactive** – code runs line by line, so testing is quick (no compilation).
5. Free, open source, huge library support (SciPy, Pandas) and also works with Raspberry Pi.

(Write any four.)

Q2 c) Boolean value and cooling fan (6 marks)

A Boolean has only two values: `True` or `False` (type `bool`). It is used in conditions.

```
temp = 80
fan_on = temp > 60      # True or False
if fan_on:
    print("Fan ON")
else:
    print("Fan OFF")
```

Q2 d) Type conversion – rod area (6 marks)

`input()` returns text; text cannot be used in arithmetic. So it is converted to a number using `float()` (explicit type conversion / type casting).

```
d = float(input("Enter diameter (mm): "))   # str -> float
area = 3.1416 * d**2 / 4
print("Cross-sectional area =", area, "mm^2")
```

SECTION II

Q3 a) Lift load 50 kg to 500 kg (7 marks)

```
load = float(input("Enter load (kg): "))
if 50 <= load <= 500:
    print("Load is permitted")
else:
    print("Load is not permitted")
```

Q3 b) break, continue, pass (6 marks)

- **break** – exits the loop immediately.
- **continue** – skips the current iteration and goes to the next.
- **pass** – does nothing; a placeholder.

```
dia = [10, 12, 0, 14, -1, 16]     # 0 or negative = defect

for d in dia:                      # break
    if d <= 0:
        print("Defect found, stop")
        break

for d in dia:                      # continue
    if d <= 0:
        continue                   # skip defective
    print("OK:", d)

for d in dia:                      # pass
    if d <= 0:
        pass                       # to be coded later
```

Q3 c) Function parameters and return values (6 marks)

- **Parameters** are variables in the function definition that receive input values (arguments).
- **Return value** is the result sent back to the caller using `return`.

```
def volume(l, w, t):
    return l * w * t

v1 = volume(200, 100, 10)
v2 = volume(150, 80, 5)
```

```
print(v1, v2)       # 200000 60000
```

Q3 d) Built-in vs user-defined functions (6 marks)

Built-in	User-defined
Already provided by Python	Written by the programmer using `def`
e.g. `max()`, `sum()`, `len()`	e.g. `volume()`
No coding needed	Coded for a specific need

```
loads = [12, 45, 30, 22]
print(max(loads), sum(loads))     # built-in: 1 line each

def my_max(lst):                  # user-defined: longer
    m = lst[0]
    for x in lst:
        if x > m:
            m = x
    return m
```

The built-in version is shorter, faster and error-free.

OR

Q4 a) Boiler temperature using elif (7 marks)

```
t = float(input("Enter temperature (C): "))
if t < 100:
    print("Low temperature")
elif t <= 180:
    print("Normal temperature")
else:
    print("High temperature - danger")
```

(Limits 100 and 180 are assumed.)

Q4 b) for loop vs while loop (6 marks)

for loop	while loop
Used when the number of repetitions is known	Used when the number of repetitions is not known; runs until a condition becomes false
Iterates over a sequence/range	Repeats while a condition is True
Counter updated automatically	Counter must be updated manually

- **for loop (better):** checking all 50 components in a batch (count known).
- **while loop (better):** heating a component until temperature reaches 500 °C (time unknown).

Q4 c) Why write own functions (6 marks)

- Built-in functions cannot do every specific job (e.g. calculating shaft torque).
- Code is written once and **reused** whenever the calculation repeats; avoids duplication.
- Program becomes shorter, organised and easier to read.
- A change/correction is made in one place only, so debugging is easier.

Q4 d) Local and global variables (6 marks)

- **Global variable:** defined outside functions; can be used by all functions.
- **Local variable:** defined inside a function; exists only inside that function.

```
g = 9.81                      # global, shared

def weight(m):
    w = m * g                 # w is local
    return w

def potential_energy(m, h):
    pe = m * g * h            # pe is local
    return pe

print(weight(10))
print(potential_energy(10, 5))
# print(w)  -> NameError: w is not defined outside the function
```

Intermediate results (`w`, `pe`) stay inside each function, while `g` is shared by all.